

DLD — CT1 Crash Revision

Number Systems • Codes • Boolean Algebra • Logic Gates

Target: 40+/50. Priority order for tonight — do sections in this order if short on time: 1) Conversions & Complements, 2) Codes, 3) Boolean Laws + De Morgan's, 4) Gates + Truth Tables, 5) Universal Gates, 6) Simplification, 7) Definitions.

1. NUMBER SYSTEM CONVERSIONS [HIGH PRIORITY]

Base values: Binary=2, Octal=8, Decimal=10, Hexadecimal=16

Decimal → Binary/Octal/Hex

Repeatedly divide by the base, note remainders bottom-to-top.

Example: $45 \rightarrow \text{Binary} \rightarrow 45 \div 2 = 22 \text{ r}1, 22 \div 2 = 11 \text{ r}0, 11 \div 2 = 5 \text{ r}1, 5 \div 2 = 2 \text{ r}1, 2 \div 2 = 1 \text{ r}0, 1 \div 2 = 0 \text{ r}1 \rightarrow \text{read bottom-up} = \mathbf{101101}$

Binary/Octal/Hex → Decimal

Multiply each digit by $(\text{base})^{\text{position}}$, position counted from 0 at rightmost.

Example: $1011 \text{ (binary)} = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = \mathbf{11}$

Example: $3A \text{ (hex)} = 3 \times 16^1 + 10 \times 16^0 = 48 + 10 = \mathbf{58}$

Binary ↔ Octal (group in 3s)

Binary→Octal: group bits in 3s from right, pad zeros on left if needed, convert each group.

Example: $101110 \rightarrow 101 \ 110 \rightarrow 5 \ 6 \rightarrow \mathbf{56 \text{ (octal)}}$

Octal→Binary: convert each octal digit to 3-bit binary and join.

Example: $47 \text{ (octal)} \rightarrow 4=100, 7=111 \rightarrow \mathbf{100111}$

Binary ↔ Hexadecimal (group in 4s)

Binary→Hex: group bits in 4s from right, pad zeros if needed, convert each group.

Example: $10110111 \rightarrow 1011 \ 0111 \rightarrow B \ 7 \rightarrow \mathbf{B7 \text{ (hex)}}$

Hex→Binary: convert each hex digit to 4-bit binary and join.

Example: $2F \text{ (hex)} \rightarrow 2=0010, F=1111 \rightarrow \mathbf{00101111}$

Octal ↔ Hexadecimal

No direct shortcut — convert Octal→Binary→Hex (or Hex→Binary→Octal) via the 3-bit/4-bit grouping above.

Hex digit reference: $A=10, B=11, C=12, D=13, E=14, F=15$

2. BINARY ARITHMETIC [HIGH PRIORITY]

Addition rules

$0+0=0$, $0+1=1$, $1+0=1$, $1+1=10$ (0 carry 1), $1+1+1=11$ (1 carry 1)

Example: $1011 + 1101$

```
1011
+ 1101
-----
11000
```

Subtraction rules

$0-0=0$, $1-0=1$, $1-1=0$, $0-1=1$ (borrow 1 from next column)

Example: $1101 - 1011 = 0010$

Tip: exam-friendly method is to add the 2's complement instead of borrowing directly (see Section 3).

Multiplication rules

Same as decimal long multiplication but with $0 \times 0 = 0$, $0 \times 1 = 0$, $1 \times 0 = 0$, $1 \times 1 = 1$, then add shifted partial products.

Example: 101×11

```
101
× 011
-----
101
101_
-----
1111
```

3. COMPLEMENTS [HIGH PRIORITY]

1's Complement (Binary)

Flip every bit: $0 \rightarrow 1$ and $1 \rightarrow 0$.

Example: $1010 \rightarrow 1\text{'s complement} = 0101$

2's Complement (Binary)

Take 1's complement, then add 1.

Example: $1010 \rightarrow 1\text{'s comp} = 0101 \rightarrow +1 = 0110$

Shortcut: from the right, keep digits unchanged until the first '1', then flip all bits after it.

9's Complement (Decimal)

Subtract each digit from 9.

Example: $4523 \rightarrow 9999 - 4523 = 5476$

10's Complement (Decimal)

Take 9's complement, then add 1.

Example: $4523 \rightarrow 9\text{'s comp} = 5476 \rightarrow +1 = 5477$

Why complements matter: used to perform subtraction via addition in digital circuits (most likely to appear as a 'why is 2's complement used' theory question).

4. CODES — WEIGHTED & NON-WEIGHTED [HIGH PRIORITY]

Weighted codes: each bit position has a fixed weight (value). Non-weighted codes: positions have no fixed weight — used for error detection or special sequencing.

Code	Type	Weights / Rule
8421 BCD	Weighted	Standard binary weights 8,4,2,1 — same as 4-bit binary per decimal digit
2421	Weighted	Weights 2,4,2,1 — self-complementing code
5211	Weighted	Weights 5,2,1,1
Excess-3	Non-weighted	Add 3 (0011) to the 8421 BCD code of the digit
Gray Code	Non-weighted	Only one bit changes between successive values

BCD (8421) table for 0–9

Decimal	0	1	2	3	4	5	6	7	8	9
8421 BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Excess-3 table for 0–9 (BCD + 0011)

Decimal	0	1	2	3	4	5	6	7	8	9
Excess-3	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100

5. CODE CONVERSIONS [HIGH PRIORITY]

Binary → Gray Code

MSB of Gray = MSB of Binary. Each next Gray bit = XOR of current and previous Binary bit.

Example: Binary 1011 → Gray: 1, $1\oplus0=1$, $0\oplus1=1$, $1\oplus1=0$ → **1110**

Gray Code → Binary

MSB of Binary = MSB of Gray. Each next Binary bit = XOR of previous Binary bit and current Gray bit.

Example: Gray 1110 → Binary: 1, $1\oplus1=0$, $0\oplus1=1$, $1\oplus0=1$ → **1011**

Decimal → BCD / BCD → Decimal

Convert each decimal digit separately to its 4-bit 8421 code (and vice versa — split into groups of 4 bits, convert each group back to decimal digit).

Example: 29 → BCD = 0010 1001

Decimal → Excess-3 / Excess-3 → Decimal

Decimal→Excess-3: get BCD of each digit, add 0011 (3) to each 4-bit group.

Excess-3→Decimal: subtract 0011 (3) from each 4-bit group, then read as BCD.

Example: 8 → BCD 1000 → $+0011 = 1011$ (**Excess-3**)

6-9. BOOLEAN ALGEBRA [HIGH PRIORITY]

Basics

- **Boolean variable:** a symbol (A, B, C...) representing a value of 0 or 1.
- **Boolean expression:** a combination of variables and operators, e.g. $A \cdot B + C'$.
- **Boolean operators:** AND ($\cdot$), OR (+), NOT ($'$ or bar) — the three fundamental operations.

Boolean Algebra Laws

Law	AND form	OR form
Commutative	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$
Distributive	$A \cdot (B + C) = A \cdot B + A \cdot C$	$A + (B \cdot C) = (A + B) \cdot (A + C)$
Identity	$A \cdot 1 = A$	$A + 0 = A$
Null (Domination)	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent	$A \cdot A = A$	$A + A = A$
Complement	$A \cdot A' = 0$	$A + A' = 1$
Absorption	$A \cdot (A + B) = A$	$A + (A \cdot B) = A$
Involution	$(A')' = A$	—

Duality Property

To find the dual of an expression: swap AND $\leftrightarrow$ OR, and swap 0 $\leftrightarrow$ 1. Variables stay unchanged.

Example: $A + (B \cdot C) \rightarrow \text{dual} \rightarrow A \cdot (B + C)$

Example: $A \cdot 1 = A \rightarrow \text{dual} \rightarrow A + 0 = A$

De Morgan's Theorems

First theorem: $(A \cdot B)' = A' + B'$ (NAND of inputs = OR of complemented inputs)

Second theorem: $(A + B)' = A' \cdot B'$ (NOR of inputs = AND of complemented inputs)

Proof approach: build a truth table for LHS and RHS separately for all combinations of A,B — if columns match exactly, the theorem is proved.

A	B	A'	B'	$(A \cdot B)'$	$A' + B'$	$(A + B)'$	$A' \cdot B'$
0	0	1	1	1	1	1	1
0	1	1	0	1	1	0	0
1	0	0	1	1	1	0	0
1	1	0	0	0	0	0	0

$(A \cdot B)'$ matches $A' + B'$ in every row $\rightarrow$ 1st theorem proved. $(A + B)'$ matches $A' \cdot B'$ $\rightarrow$ 2nd theorem proved.

Application: De Morgan's laws are used to convert AND/OR circuits into NAND/NOR-only circuits, and to simplify complemented expressions before implementation.

10-13. LOGIC GATES [HIGH PRIORITY]

Basic Gates: AND, OR, NOT

Gate	Expression	Truth Table (Inputs→Output)	Real-life use
AND	$Y = A \cdot B$	00→0, 01→0, 10→0, 11→1	Series switch circuit — both conditions must be true
OR	$Y = A + B$	00→0, 01→1, 10→1, 11→1	Parallel switch circuit — either condition triggers output
NOT	$Y = A'$	0→1, 1→0	Inverter — used to complement a signal

Derived Gates: NAND, NOR, XOR, XNOR

Gate	Expression	Truth Table (Inputs→Output)	Real-life use
NAND	$Y = (A \cdot B)'$	00→1, 01→1, 10→1, 11→0	Universal gate — used to build all other gates
NOR	$Y = (A + B)'$	00→1, 01→0, 10→0, 11→0	Universal gate — used in memory (SR latch)
XOR	$Y = A \oplus B$	00→0, 01→1, 10→1, 11→0	Adders, comparators, parity/error-detection circuits
XNOR	$Y = (A \oplus B)'$	00→1, 01→0, 10→0, 11→1	Equality checkers/comparators

Symbols: draw AND as a 'D' shape, OR as a curved-back shield shape, NOT as a triangle with a bubble at the tip. NAND/NOR/XOR/XNOR = AND/OR/XOR shape + a bubble at output (XOR/XNOR also have a curved double line at input).

Basic vs Derived Gates

- **Basic gates:** AND, OR, NOT — the fundamental building blocks, cannot be broken down further.
- **Derived gates:** NAND, NOR, XOR, XNOR — built by combining basic gates.

Universal Gates: NAND & NOR

A gate is 'universal' if ANY logic gate (AND, OR, NOT) can be built using only that gate.

Target Gate	Using only NAND	Using only NOR
NOT (A')	Tie both NAND inputs to A	Tie both NOR inputs to A
AND ($A \cdot B$)	NAND(A,B) then NOT that output	NOR(NOT A, NOT B)
OR ($A + B$)	NOT A, NOT B, then NAND them	NOR(A,B) then NOT that output

Memory trick: NOT = self-fed gate. AND = NAND+NOT. OR = NOT the inputs first, then NAND (for NAND-only OR) — this is essentially De Morgan's theorem in circuit form.

14-16. CIRCUITS, TRUTH TABLES & SIMPLIFICATION [PRIORITY]

Logic diagram $\leftrightarrow$ Boolean expression

- To draw a circuit from an expression: work from innermost brackets outward, one gate per operation.
- To write an expression from a circuit: trace signal flow from inputs to final output, naming each gate's operation.
- Always match the number of inputs on each gate to the number of terms feeding it.

Truth Table Construction

- For n variables, there are 2^n rows — list all combinations in binary order (00...0 to 11...1).
- Build intermediate columns for sub-expressions before the final output column — reduces mistakes.
- To verify an equivalence, build truth tables for both sides and check every row matches (as done for De Morgan's above).

Boolean Expression Simplification — worked example

Simplify: $Y = A \cdot B + A \cdot B' + A' \cdot B$

$$Y = A(B+B') + A'B \quad [\text{Distributive}]$$

$$Y = A(1) + A'B \quad [\text{Complement law: } B+B'=1]$$

$$Y = A + A'B \quad [\text{Identity law}]$$

$$Y = A + B \quad [\text{Absorption-type identity: } A+A'B = A+B]$$

Simplify using De Morgan's: $Y = (A'+B)'$

$$Y = (A')' \cdot (B')' = A \cdot B \quad [\text{Apply De Morgan's, then Involution law}]$$

17. QUICK-FIRE DEFINITIONS [PRIORITY]

Truth table: A table listing all possible input combinations of a logic circuit and their corresponding outputs.

Derived gates: Logic gates built by combining basic gates (AND, OR, NOT) — e.g. NAND, NOR, XOR, XNOR.

Weighted code: A code where each bit position carries a fixed numerical weight, e.g. 8421 BCD.

Non-weighted code: A code where bit positions carry no fixed weight, e.g. Gray code, Excess-3.

Duality property: Every Boolean identity remains valid if $AND \leftrightarrow OR$ and $0 \leftrightarrow 1$ are interchanged throughout.

Universal gate: A gate (NAND or NOR) that can be used alone to implement any other logic gate/function.

Boolean algebra: A mathematical system dealing with binary variables and logical operations (AND, OR, NOT), used to analyze/simplify digital circuits.

Logic gate: An electronic circuit that performs a basic Boolean operation on one or more binary inputs to produce a single binary output.

Final tip: In DLD papers, conversions/complements/codes are usually fast, guaranteed marks if you drill 5-10 practice problems tonight. Boolean laws + De Morgan's + gates are usually a mix of 'state and prove' and 'simplify this expression' — memorize the law table and the worked example pattern above and you can adapt it to any similar expression in the exam.